```
# EXPERIMENT NO. 10
# Character, Digit, or Face Classification using Convolutional Neural Networks
import os
import shutil
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# 1. EXPERIMENT INFORMATION
print("=" * 80)
print("EXPERIMENT NO. 10")
print("Character, Digit, or Face Classification using CNNs")
print("=" * 80)

print("\nDataset Selected: MNIST Handwritten Digit Dataset")
print("Task: Handwritten Digit Classification")
print("Classes: 0 to 9")

# 2. CREATE RESULT FOLDER
RESULT_DIR = "/content/Experiment_10_Results"
if os.path.exists(RESULT_DIR):
    shutil.rmtree(RESULT_DIR)
os.makedirs(RESULT_DIR)

# 3. LOAD MNIST DATASET
print("\n📥 Loading MNIST Dataset...")
(x_train_full, y_train_full), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
print("✅ MNIST Dataset Loaded Successfully")
print("\nOriginal Dataset Information")
print("-" * 60)
print("Training Images :", x_train_full.shape)
print("Training Labels :", y_train_full.shape)
print("Testing Images  :", x_test.shape)
print("Testing Labels  :", y_test.shape)

# 4. DATA PREPROCESSING
print("\n🔄 Performing Image Preprocessing...")
# Normalize pixel values from 0-255 to 0-1
x_train_full = x_train_full.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

```
# Add channel dimension
x_train_full = np.expand_dims(x_train_full, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)

# Create validation set
x_val = x_train_full[-6000:]
y_val = y_train_full[-6000:]
x_train = x_train_full[:-6000]
y_train = y_train_full[:-6000]
print("Training Set   :", x_train.shape)
print("Validation Set :", x_val.shape)
print("Testing Set    :", x_test.shape)
print("\nPixel Range After Normalization:")
print("Minimum:", x_train.min())
print("Maximum:", x_train.max())

# 5. DISPLAY SAMPLE IMAGES
plt.figure(figsize=(12, 6))
for i in range(12):
    plt.subplot(3, 4, i + 1)
    plt.imshow(x_train[i].squeeze(), cmap="gray")
    plt.title(f"Label: {y_train[i]}")
    plt.axis("off")
plt.suptitle("Sample Images from MNIST Dataset")
plt.tight_layout()
plt.savefig(
    os.path.join(RESULT_DIR, "01_MNIST_Sample_Images.png"),
    dpi=200,
    bbox_inches="tight"
)
plt.show()

# 6. DESIGN CNN MODEL
print("\n🧠 Building Convolutional Neural Network...")
model = models.Sequential([
    # Input Layer
    layers.Input(shape=(28, 28, 1)),
    # Convolutional Layer 1
    layers.Conv2D(
        32,
        (3, 3),
        activation="relu",
        padding="same"
    ),
# Pooling Layer 1
    layers.MaxPooling2D((2, 2)),
    # Convolutional Layer 2
    layers.Conv2D(
        64,
```

```
        (3, 3),
        activation="relu",
        padding="same"
    ),
    # Pooling Layer 2
    layers.MaxPooling2D((2, 2)),
    # Convolutional Layer 3
    layers.Conv2D(
        128,
        (3, 3),
        activation="relu",
        padding="same"
    ),

    # Flatten
    layers.Flatten(),

    # Fully Connected Layer
    layers.Dense(
        128,
        activation="relu"
    ),

    # Output Layer - 10 Classes
    layers.Dense(
        10,
        activation="softmax"
    )
])
print("\n📐 CNN Architecture:")
model.summary()
# Save model architecture summary
with open(
    os.path.join(RESULT_DIR, "02_CNN_Model_Summary.txt"),
    "w"
) as f:
    model.summary(print_fn=lambda x: f.write(x + "\n"))

# 7. COMPILE CNN MODEL
print("\n⚙️ Compiling CNN Model...")
model.compile(
    optimizer=tf.keras.optimizers.Adam(
        learning_rate=0.001
    ),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
print("Optimizer       : Adam")
print("Learning Rate   : 0.001")
print("Loss Function   : Sparse Categorical Crossentropy")
```

```
print("Batch Size      : 128")
print("Epochs          : 5")

# 8. TRAIN CNN MODEL
print("\n🚀 Training CNN Model...")
print("=" * 80)
history = model.fit(
    x_train,
    y_train,
    validation_data=(x_val, y_val),
    epochs=5,
    batch_size=128,
    verbose=1
)
print("\n✅ CNN Training Completed Successfully")

# 9. TRAINING AND VALIDATION ACCURACY
plt.figure(figsize=(10, 6))
plt.plot(
    history.history["accuracy"],
    marker="o",
    label="Training Accuracy"
)
plt.plot(
    history.history["val_accuracy"],
    marker="o",
    label="Validation Accuracy"
)
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig(
    os.path.join(
        RESULT_DIR,
        "03_Training_Validation_Accuracy.png"
    ),
    dpi=200,
    bbox_inches="tight"
)
plt.show()

# 10. TRAINING AND VALIDATION LOSS
plt.figure(figsize=(10, 6))
plt.plot(
    history.history["loss"],
    marker="o",
    label="Training Loss"
```

```
)
plt.plot(
    history.history["val_loss"],
    marker="o",
    label="Validation Loss"
)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig(
    os.path.join(
        RESULT_DIR,
        "04_Training_Validation_Loss.png"
    ),
    dpi=200,
    bbox_inches="tight"
)
plt.show()

# 11. EVALUATE MODEL ON TEST DATA
print("\n📊 Evaluating CNN on Test Dataset...")
test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)
print("\nTEST RESULTS")
print("-" * 60)
print("Test Loss     :", round(test_loss, 4))
print("Test Accuracy :", round(test_accuracy * 100, 2), "%")

# 12. PREDICT TEST DATA
print("\n🔮 Generating Predictions...")
predicted_probabilities = model.predict(
    x_test,
    verbose=0
)
y_pred = np.argmax(
    predicted_probabilities,
    axis=1
)

# 13. CALCULATE PERFORMANCE METRICS
accuracy = accuracy_score(
    y_test,
    y_pred
)
```

```
precision = precision_score(
    y_test,
    y_pred,
    average="weighted",
    zero_division=0
)
recall = recall_score(
    y_test,
    y_pred,
    average="weighted",
    zero_division=0
)
f1 = f1_score(
    y_test,
    y_pred,
    average="weighted",
    zero_division=0
)
print("\n CLASSIFICATION PERFORMANCE")
print("=" * 60)
print(f"Accuracy  : {accuracy * 100:.2f}%")
print(f"Precision : {precision * 100:.2f}%")
print(f"Recall    : {recall * 100:.2f}%")
print(f"F1-Score  : {f1 * 100:.2f}%")

# 14. SAVE PERFORMANCE METRICS
metrics_df = pd.DataFrame({
    "Metric": [
        "Accuracy",
        "Precision",
        "Recall",
        "F1-Score"
    ],
 "Score": [
        round(accuracy, 4),
        round(precision, 4),
        round(recall, 4),
        round(f1, 4)    ],
 "Percentage": [
        round(accuracy * 100, 2),
        round(precision * 100, 2),
        round(recall * 100, 2),
        round(f1 * 100, 2)
    ]
})
metrics_df.to_csv(
    os.path.join(
        RESULT_DIR,
        "05_Performance_Metrics.csv"
    ),
```

```
    index=False
)
print("\n")
print(metrics_df.to_string(index=False))

# 15. PERFORMANCE METRICS GRAPH
plt.figure(figsize=(9, 6))
plt.bar(
    metrics_df["Metric"],
    metrics_df["Percentage"]
)
plt.ylabel("Score (%)")
plt.xlabel("Evaluation Metric")
plt.title("CNN Classification Performance")
plt.ylim(0, 100)
for i, value in enumerate(metrics_df["Percentage"]):
    plt.text(
        i,
        value + 1,
        f"{value:.2f}%",
        ha="center"
    )
plt.tight_layout()
plt.savefig(
    os.path.join(
        RESULT_DIR,
        "06_Performance_Metrics_Graph.png"
    ),
    dpi=200,
    bbox_inches="tight"
)
plt.show()

# 16. CONFUSION MATRIX
print("\n🔲 Generating Confusion Matrix...")
cm = confusion_matrix( y_test,y_pred )
plt.figure(figsize=(9, 8))
plt.imshow(cm)
plt.title("Confusion Matrix - MNIST CNN")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.xticks(
    range(10),
    range(10)
)
plt.yticks(
    range(10),
    range(10)
)
plt.colorbar()
```

```
for i in range(10):
    for j in range(10):
        plt.text(
            j,
            i,
            cm[i, j],
            ha="center",
            va="center"
        )
plt.tight_layout()
plt.savefig(
    os.path.join(
        RESULT_DIR,
        "07_Confusion_Matrix.png"
    ),
    dpi=200,
    bbox_inches="tight"
)
plt.show()
# Save confusion matrix CSV
cm_df = pd.DataFrame(
    cm,
    index=[f"Actual_{i}" for i in range(10)],
    columns=[f"Predicted_{i}" for i in range(10)]
)
cm_df.to_csv(
    os.path.join(
        RESULT_DIR,
        "08_Confusion_Matrix.csv"
    )
)
# 17. CLASSIFICATION REPORT
report = classification_report(y_test,y_pred, digits=4)
print("\n CLASSIFICATION REPORT")
print("=" * 80)
print(report)

with open(
    os.path.join(
        RESULT_DIR,
        "09_Classification_Report.txt"
    ),
    "w"
) as f:
    f.write(report)
# 18. PREDICTION ON UNSEEN TEST IMAGES
print("\n Predictions on Unseen Images")
sample_indices = np.random.choice(
    len(x_test),12,   replace=False)
plt.figure(figsize=(12, 8))
```

```
prediction_rows = []
for plot_index, image_index in enumerate(sample_indices):
    image_sample = x_test[image_index]
    prediction = model.predict(
        np.expand_dims(
            image_sample,
            axis=0
        ),
        verbose=0
    )
    predicted_digit = np.argmax(prediction)
    confidence = np.max(prediction)
    actual_digit = y_test[image_index]
    prediction_rows.append({
        "Actual": actual_digit,
        "Predicted": predicted_digit,
        "Confidence": round(float(confidence), 4),
        "Correct": actual_digit == predicted_digit
    })
    plt.subplot(3, 4, plot_index + 1)
    plt.imshow(
        image_sample.squeeze(),
        cmap="gray"
    )
    plt.title(
        f"Actual: {actual_digit}\n"
        f"Predicted: {predicted_digit}\n"
        f"Conf: {confidence:.2f}"
    )
    plt.axis("off")
plt.suptitle(
    "CNN Predictions on Unseen MNIST Images"
)
plt.tight_layout()
plt.savefig(
    os.path.join(
        RESULT_DIR,
        "10_Unseen_Image_Predictions.png"
    ),
    dpi=200,
    bbox_inches="tight"
)

plt.show()
prediction_df = pd.DataFrame(
    prediction_rows
)
prediction_df.to_csv(
    os.path.join(
        RESULT_DIR,
```

```
        "11_Unseen_Image_Predictions.csv"
    ),
    index=False
)

print(prediction_df.to_string(index=False))

# 19. MISCLASSIFIED IMAGES
print("\n❌ Analyzing Misclassified Images...")
wrong_indices = np.where(
    y_pred != y_test
)[0]
print(
    "Total Misclassified Images:",
    len(wrong_indices)
)
plt.figure(figsize=(12, 8))
number_to_display = min(
    12,
    len(wrong_indices)
)
for i in range(number_to_display):
    index = wrong_indices[i]
    plt.subplot(3, 4, i + 1)
    plt.imshow(
        x_test[index].squeeze(),
        cmap="gray"
    )
    plt.title(
        f"Actual: {y_test[index]}\n"
        f"Predicted: {y_pred[index]}"
    )
plt.axis("off")
plt.suptitle("Examples of Misclassified MNIST Images")
plt.tight_layout()
plt.savefig(
    os.path.join(
        RESULT_DIR,
        "12_Misclassified_Images.png"
    ),
    dpi=200,
    bbox_inches="tight"
)
plt.show()

# 20. MODEL PERFORMANCE SUMMARY
summary_df = pd.DataFrame({
    "Parameter": ["Dataset","Number of Classes","Image Size","Training Images","Validation
Images","Testing Images","CNN Convolutional Layers","Optimizer","Learning Rate","Batch
```

```
Size","Epochs","Test Accuracy","Test Loss","Precision","Recall",  "F1-Score",
"Misclassified Images"
],

"Value": ["MNIST",10,"28 x 28", len(x_train), len(x_val),len(x_test), 3, "Adam" ,0.001,
128,5,f"{accuracy * 100:.2f}%",round(test_loss, 4),f"{precision * 100:.2f}%",f"{recall *
100:.2f}%", f"{f1 * 100:.2f}%",len(wrong_indices)
 ]
})
summary_df.to_csv(
    os.path.join(
        RESULT_DIR,
        "13_Final_Experiment_Summary.csv"
    ),
    index=False
)
print("\n FINAL EXPERIMENT SUMMARY")
print("=" * 80)
print(summary_df.to_string(index=False))

# 21. FINAL OBSERVATIONS
print("\n")
print("=" * 80)
print("FINAL OBSERVATIONS")
print("=" * 80)
print(f"""
1. The MNIST handwritten digit dataset was successfully loaded and preprocessed.
2. Pixel values were normalized from the range 0-255 to 0-1, which helps
   stabilize and improve neural network training.
3. A Convolutional Neural Network containing convolutional, pooling,
   flattening, fully connected and softmax output layers was designed.
4. The CNN automatically learned important visual features from handwritten
   digit images without requiring handcrafted feature extraction.
5. The model was trained using the Adam optimizer with a learning rate of
   0.001 and a batch size of 128 for 5 epochs.
6. Training and validation accuracy/loss curves were generated to analyze
   the learning behavior of the CNN.
7. The trained model achieved a test accuracy of {accuracy * 100:.2f}% on
   the MNIST test dataset.
8. The model was evaluated using Accuracy, Precision, Recall, F1-Score and
   Confusion Matrix.
9. Unseen test images were classified successfully and their predicted labels
   were compared with the actual labels.
10. Misclassified samples were visualized to understand the limitations of
    the CNN and the similarity between some handwritten digit patterns.
11. CNN-based image classification can be applied to handwritten digit
    recognition, character recognition, document processing, biometric
    systems, medical image analysis and many other computer vision tasks.
""")
```

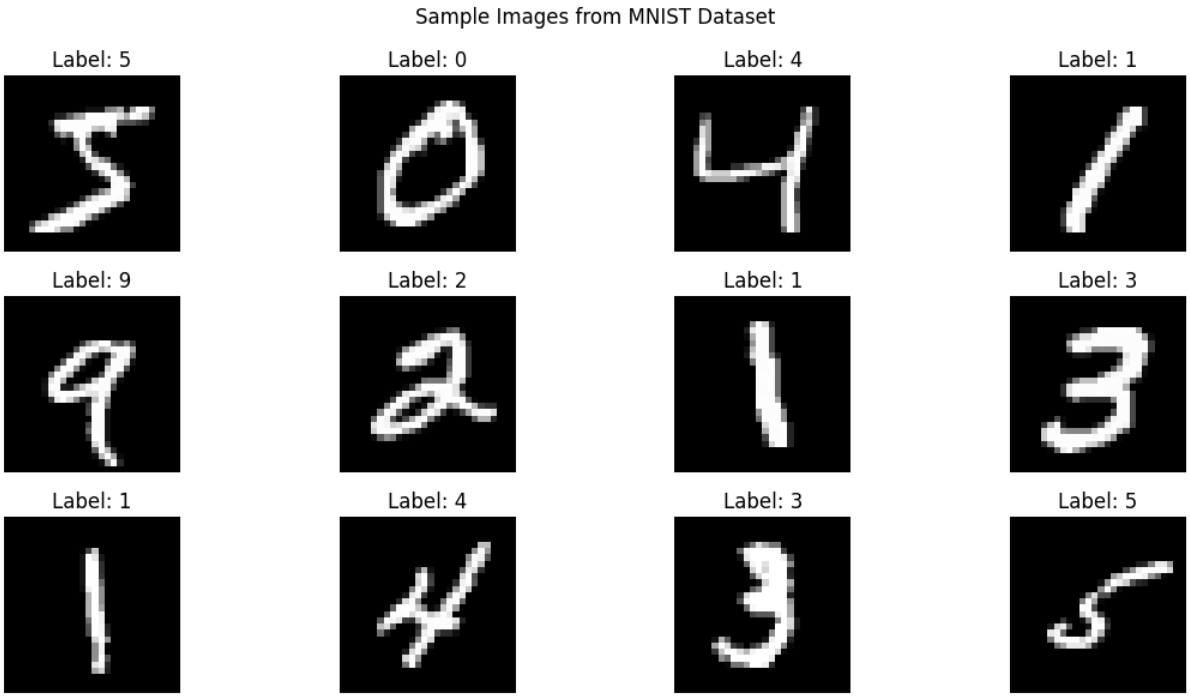
```
# 22. LIST ALL GENERATED FILES
print("\n📁 GENERATED OUTPUT FILES")
print("=" * 80)
for file_name in sorted(os.listdir(RESULT_DIR)):
    file_path = os.path.join(
        RESULT_DIR,
        file_name
    )
    size_kb = os.path.getsize(file_path) / 1024
    print(
        f"✓ {file_name:<45} "
        f"{size_kb:.2f} KB"
    )
print("\n" + "=" * 80)
print("✅ EXPERIMENT NO. 10 COMPLETED SUCCESSFULLY")
print("=" * 80)
print("\n📂 All results saved in:")
print(RESULT_DIR)
```

OUTPUTS –

EXPERIMENT NO. 10
Character, Digit, or Face Classification using CNNs
Dataset Selected: MNIST Handwritten Digit Dataset
Task: Handwritten Digit Classification
Classes: 0 to 9
📥 Loading MNIST Dataset...
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ **0s** 0us/step
✅ MNIST Dataset Loaded Successfully
Original Dataset Information
Training Images : (60000, 28, 28)
Training Labels : (60000,)
Testing Images : (10000, 28, 28)
Testing Labels : (10000,)

🔄 Performing Image Preprocessing...
Training Set   : (54000, 28, 28, 1)
Validation Set : (6000, 28, 28, 1)
Testing Set   : (10000, 28, 28, 1)

Pixel Range After Normalization:
Minimum: 0.0
Maximum: 1.0



🧠 Building Convolutional Neural Network...

📐 CNN Architecture:
Model: "sequential"

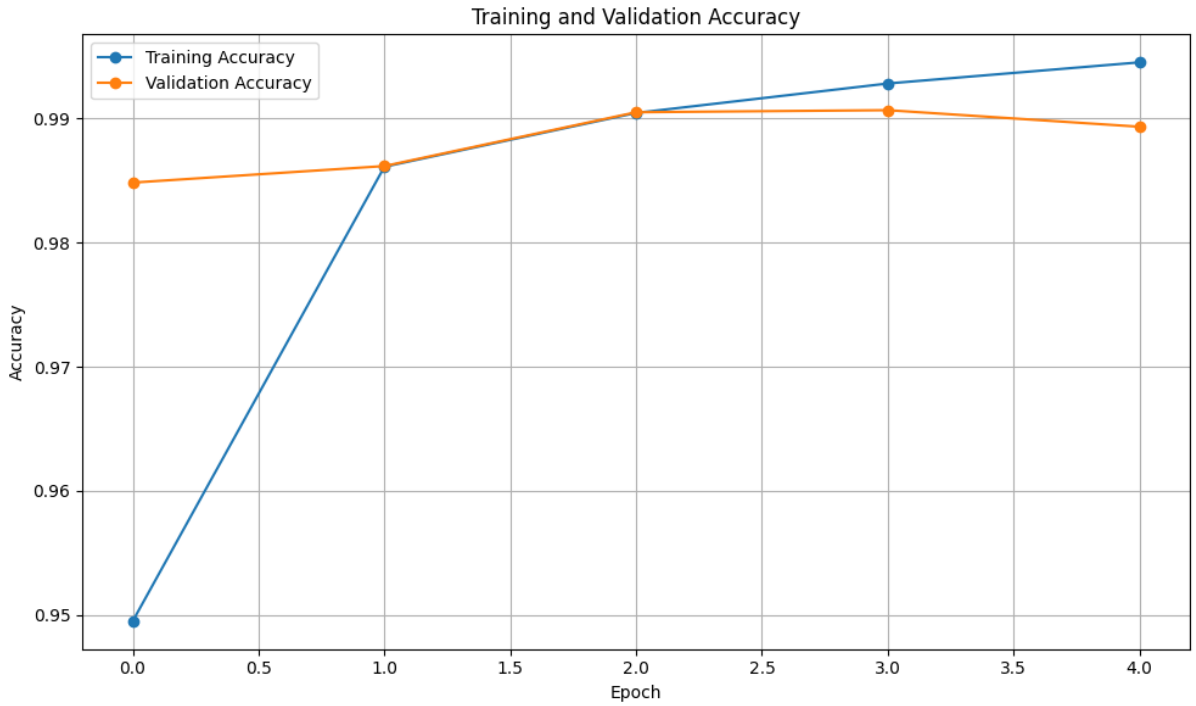
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 14, 14, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_2 (Conv2D)	(None, 7, 7, 128)	73,856
flatten (Flatten)	(None, 6272)	0
dense (Dense)	(None, 128)	802,944
dense_1 (Dense)	(None, 10)	1,290

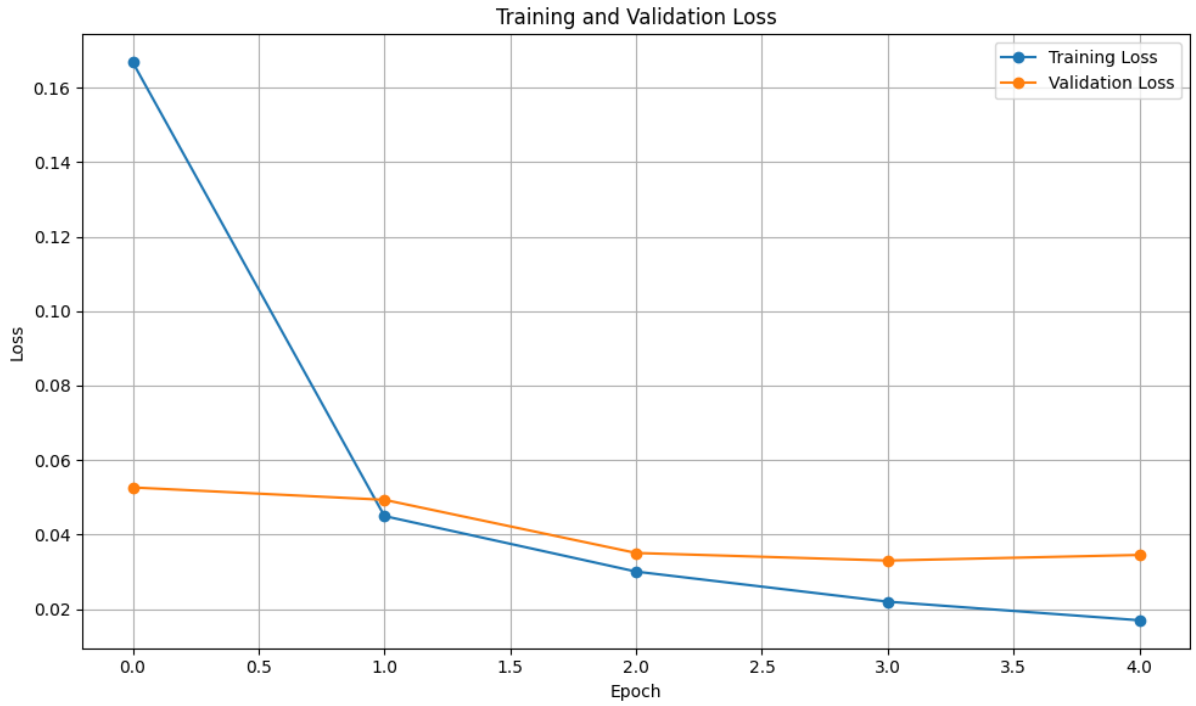
Total params: 896,906 (3.42 MB)
Trainable params: 896,906 (3.42 MB)
Non-trainable params: 0 (0.00 B)

⚙️ Compiling CNN Model...
Optimizer : Adam
Learning Rate : 0.001
Loss Function : Sparse Categorical Crossentropy
Batch Size : 128
Epochs : 5

🚀 Training CNN Model...
Epoch 1/5
422/422 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **90s** 205ms/step - accuracy: 0.9495 - loss: 0.1669 - val_accuracy: 0.9848 - val_loss: 0.0526
Epoch 2/5
422/422 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **82s** 194ms/step - accuracy: 0.9861 - loss: 0.0450 - val_accuracy: 0.9862 - val_loss: 0.0493
Epoch 3/5
422/422 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **140s** 190ms/step - accuracy: 0.9904 - loss: 0.0300 - val_accuracy: 0.9905 - val_loss: 0.0350
Epoch 4/5
422/422 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **80s** 190ms/step - accuracy: 0.9928 - loss: 0.0220 - val_accuracy: 0.9907 - val_loss: 0.0330
Epoch 5/5
422/422 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **81s** 188ms/step - accuracy: 0.9945 - loss: 0.0170 - val_accuracy: 0.9893 - val_loss: 0.0345

✅ CNN Training Completed Successfully





📊 Evaluating CNN on Test Dataset...

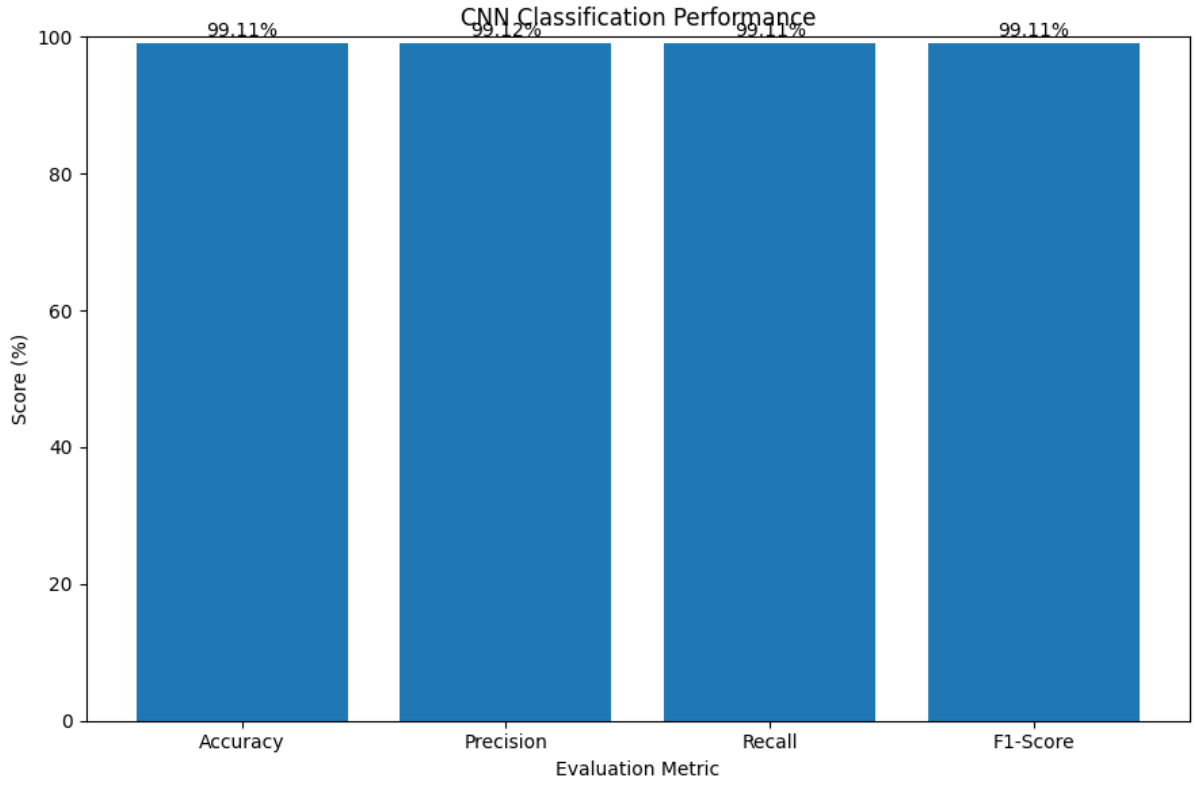
TEST RESULTS

Test Loss : 0.0279
Test Accuracy : 99.11 %

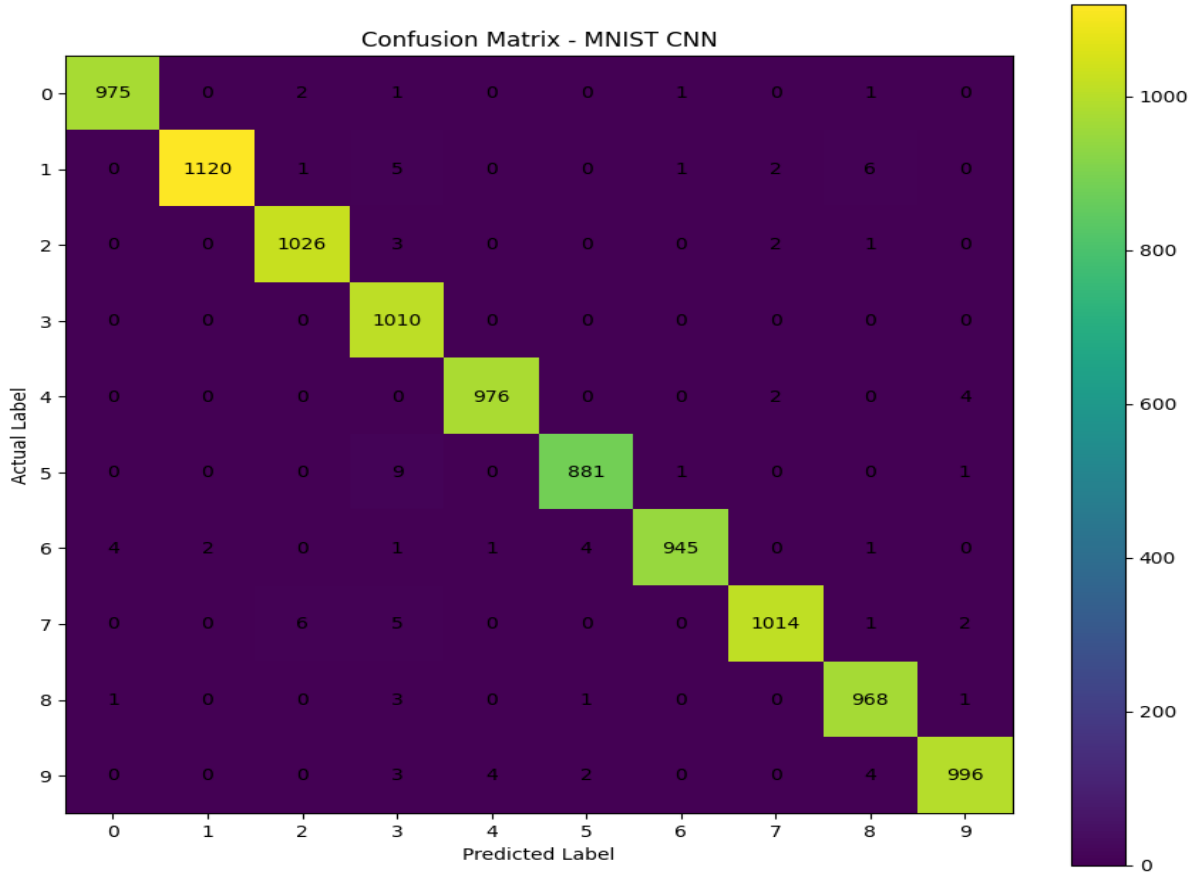
🔮 Generating Predictions...

📈 CLASSIFICATION PERFORMANCE
Accuracy : 99.11%
Precision : 99.12%
Recall : 99.11%
F1-Score : 99.11%

Metric	Score	Percentage
Accuracy	0.9911	99.11
Precision	0.9912	99.12
Recall	0.9911	99.11
F1-Score	0.9911	99.11



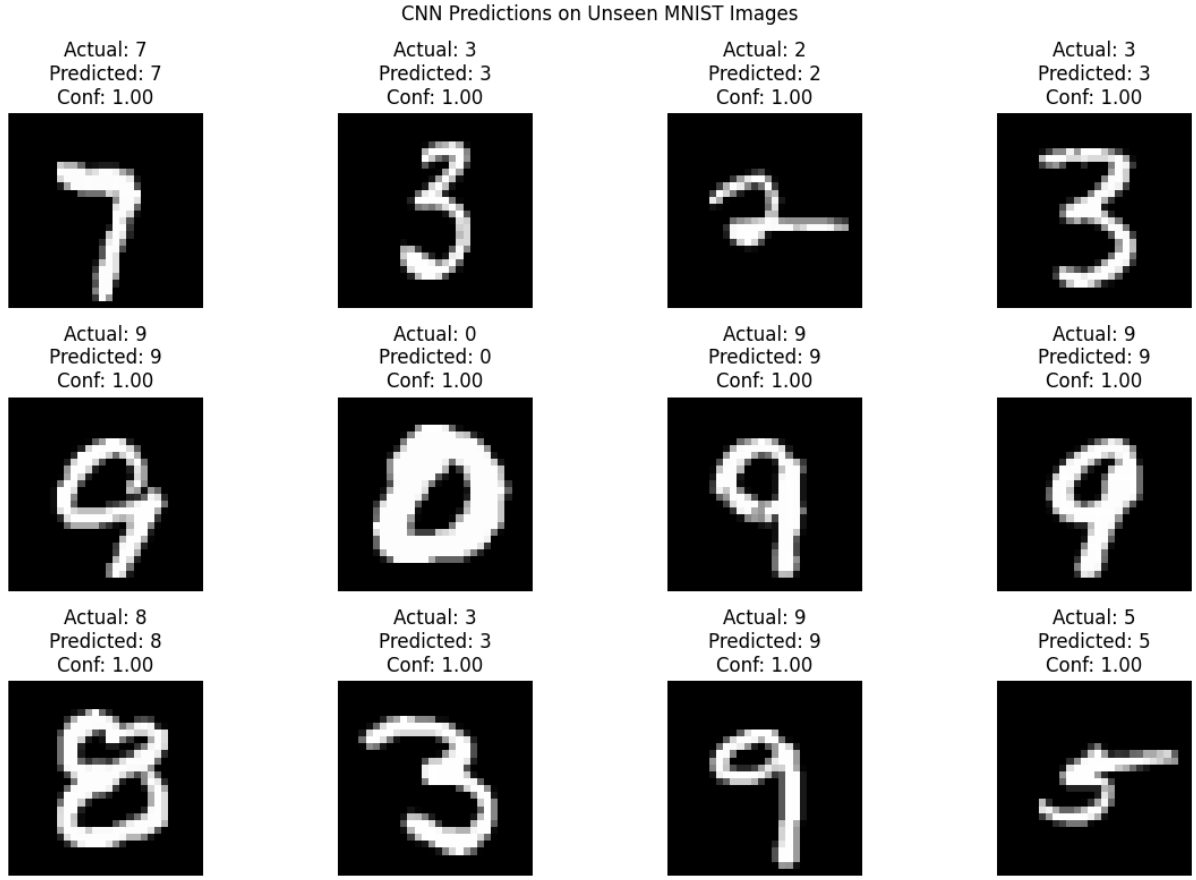
🔲 Generating Confusion Matrix...



📋 CLASSIFICATION REPORT

	precision	recall	f1-score	support
0	0.9949	0.9949	0.9949	980
1	0.9982	0.9868	0.9925	1135
2	0.9913	0.9942	0.9927	1032
3	0.9712	1.0000	0.9854	1010
4	0.9949	0.9939	0.9944	982
5	0.9921	0.9877	0.9899	892
6	0.9968	0.9864	0.9916	958
7	0.9941	0.9864	0.9902	1028
8	0.9857	0.9938	0.9898	974
9	0.9920	0.9871	0.9896	1009
accuracy			0.9911	10000
macro avg	0.9911	0.9911	0.9911	10000
weighted avg	0.9912	0.9911	0.9911	10000

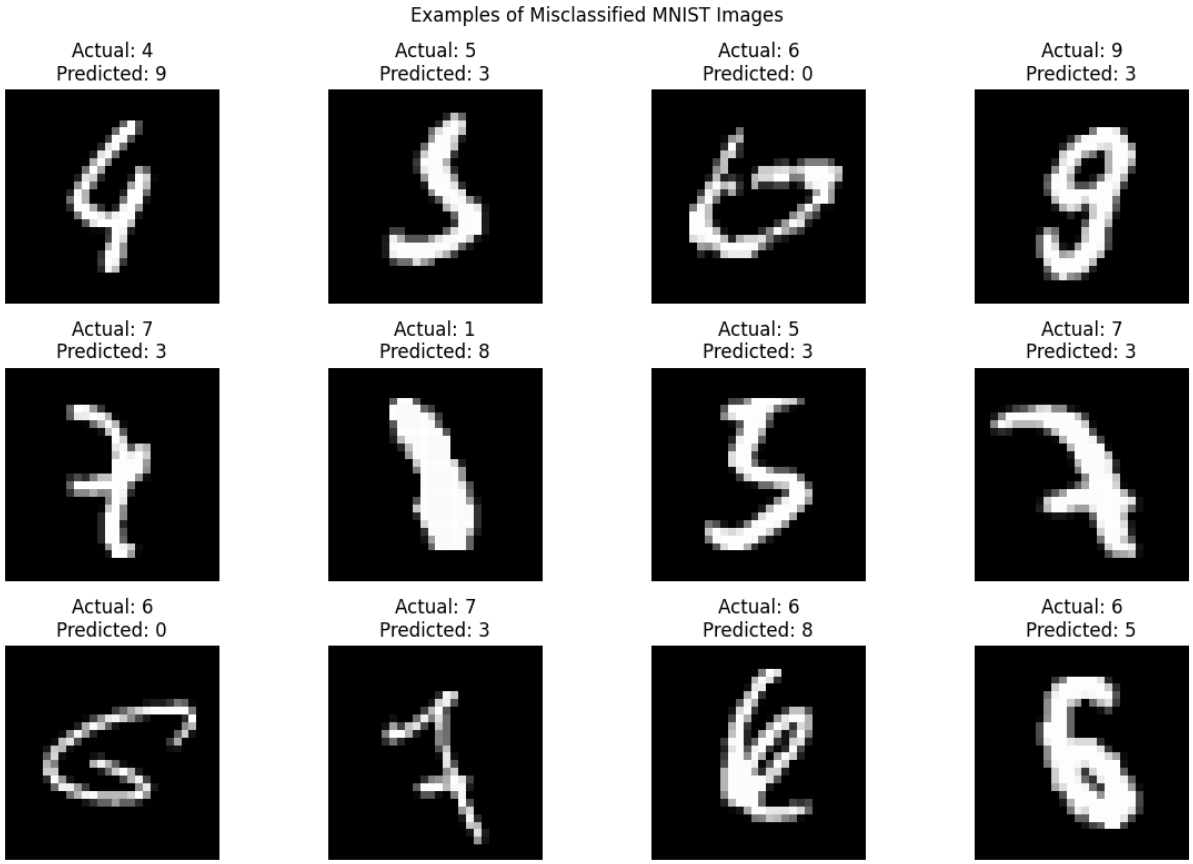
🔮 Predictions on Unseen Images



Actual	Predicted	Confidence	Correct
7	7	0.9980	True
3	3	1.0000	True
2	2	0.9991	True
3	3	1.0000	True
9	9	0.9996	True

0	0	0.9998	True
9	9	0.9966	True
9	9	0.9996	True
8	8	0.9999	True
3	3	1.0000	True
9	9	1.0000	True
5	5	0.9986	True

❌ Analyzing Misclassified Images...
Total Misclassified Images: 89



📊 FINAL EXPERIMENT SUMMARY

Parameter	Value
Dataset	MNIST
Number of Classes	10
Image Size	28 x 28
Training Images	54000
Validation Images	6000
Testing Images	10000
CNN Convolutional Layers	3
Optimizer	Adam
Learning Rate	0.001
Batch Size	128
Epochs	5
Test Accuracy	99.11%
Test Loss	0.0279

Precision	99.12%
Recall	99.11%
F1-Score	99.11%
Misclassified Images	89

FINAL OBSERVATIONS

1. The MNIST handwritten digit dataset was successfully loaded and preprocessed.
2. Pixel values were normalized from the range 0-255 to 0-1, which helps stabilize and improve neural network training.
3. A Convolutional Neural Network containing convolutional, pooling, flattening, fully connected and softmax output layers was designed.
4. The CNN automatically learned important visual features from handwritten digit images without requiring handcrafted feature extraction.
5. The model was trained using the Adam optimizer with a learning rate of 0.001 and a batch size of 128 for 5 epochs.
6. Training and validation accuracy/loss curves were generated to analyze the learning behavior of the CNN.
7. The trained model achieved a test accuracy of 99.11% on the MNIST test dataset.
8. The model was evaluated using Accuracy, Precision, Recall, F1-Score and Confusion Matrix.
9. Unseen test images were classified successfully and their predicted labels were compared with the actual labels.
10. Misclassified samples were visualized to understand the limitations of the CNN and the similarity between some handwritten digit patterns.
11. CNN-based image classification can be applied to handwritten digit recognition, character recognition, document processing, biometric systems, medical image analysis and many other computer vision tasks.

📁 GENERATED OUTPUT FILES

✓ 01_MNIST_Sample_Images.png	45.88 KB
✓ 02_CNN_Model_Summary.txt	03.11 KB
✓ 03_Training_Validation_Accuracy.png	86.66 KB
✓ 04_Training_Validation_Loss.png	91.62 KB
✓ 05_Performance_Metrics.csv	00.11 KB
✓ 06_Performance_Metrics_Graph.png	51.27 KB
✓ 07_Confusion_Matrix.png	93.81 KB
✓ 08_Confusion_Matrix.csv	00.42 KB
✓ 09_Classification_Report.txt	00.74 KB
✓ 10_Unseen_Image_Predictions.png	86.34 KB
✓ 11_Unseen_Image_Predictions.csv	00.21 KB
✓ 12_Misclassified_Images.png	74.67 KB
✓ 13_Final_Experiment_Summary.csv	0.32 KB

✅ EXPERIMENT NO. 10 COMPLETED SUCCESSFULLY
📁 All results saved in:
/content/Experiment_10_Results